

Java OOP Mastery - Phase 3

Inheritance and Polymorphism

A concept-first reference based on the Phase 3 session: inheritance, super(), overriding, upcasting, downcasting, instanceof, dynamic method dispatch, and the field/static-method traps.

1. Why Inheritance Exists

Inheritance lets a more specific class build on a more general class. The important design relationship is IS-A, not merely code copying.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

Dog IS-A Animal. Dog gets access to the inheritable members of Animal and can add specialized behavior such as bark().

2. Core Vocabulary

Term	Meaning
Superclass / Parent	The more general class, e.g. Animal
Subclass / Child	The more specific class, e.g. Dog
extends	Declares a class inheritance relationship
IS-A	The conceptual relationship represented by inheritance
HAS-A	A relationship represented by composition/association, not inheritance

3. Constructor Flow in Inheritance

```
class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    Dog() {
        System.out.println("Dog constructor");
    }
}
```

```
Dog d = new Dog();
```

Conceptually, Java initializes the parent part before the child part. When applicable, Java inserts an implicit super() as the first statement of the child constructor.

```
new Dog()
-> parent initialization
-> Animal()
-> child initialization
-> Dog()
```

4. super() and this()

this(...) delegates to another constructor in the SAME class. super(...) invokes a constructor of the IMMEDIATE parent class. A constructor can explicitly begin with one or the other, not both.

```
class Student {
    Student() {
        this("Unknown");
    }

    Student(String name) {
        System.out.println(name);
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog");
    }
}
```

5. Inherited Members and Access

A child does not get unrestricted access to every parent member. private members are not directly accessible from the child. protected members are accessible in the same package and through the appropriate subclass relationship. public members are broadly accessible.

```
class Animal {
    private int secret;
    protected String name;
    public void eat() {}
}

class Dog extends Animal {
    void test() {
        // secret = 10;    // ERROR: private
        name = "Bruno";    // OK
        eat();             // OK
    }
}
```

6. Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog");
    }
}
```

Overriding means a subclass provides a compatible implementation of an inherited instance method with the same signature. `@Override` is not what creates overriding; it tells the compiler to verify that an override is intended.

7. Overloading vs Overriding

Overloading	Overriding
Same method name, different parameter list	Same signature in parent and child
Usually resolved at compile time	For instance methods, implementation is selected at runtime
Example: <code>sound()</code> and <code>sound(int)</code>	Example: <code>Animal.sound()</code> and <code>Dog.sound()</code>

Other overriding rules: child method cannot reduce visibility; a final method cannot be overridden; constructors cannot be overridden; static methods are hidden rather than dynamically overridden; private parent methods are not overridden.

8. Upcasting - Parent Reference, Child Object

```
Animal a = new Dog();
```

Reference type = `Animal`. Actual object type = `Dog`. This assignment is implicit because `Dog` IS-A `Animal`.

9. Compile Time vs Runtime

At compile time, the compiler checks which members are available through the reference type. At runtime, an overridden instance method is dispatched according to the actual object.

```
class Animal {
    void sound() { System.out.println("Animal"); }
    void eat()   { System.out.println("Eat"); }
}

class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog"); }
    void bark()  { System.out.println("Bark"); }
}

Animal a = new Dog();
a.eat(); // allowed: Animal declares/inherits eat()
a.sound(); // allowed, runtime calls Dog.sound()
// a.bark(); // compile-time error: Animal reference has no bark()
```

Master rule: reference type controls accessible members; for overridden instance methods, the actual object controls which implementation executes.

10. Downcasting

```
Animal a = new Dog();
Dog d = (Dog) a;
d.bark();
```

Downcasting converts a parent-typed reference back to a child reference. It is safe only when the actual object really is the child type (or compatible subtype).

```
Animal a = new Cat();
Dog d = (Dog) a; // runtime failure
// ClassCastException
```

11. instanceof

```
if (a instanceof Dog) {
    Dog d = (Dog) a;
    d.bark();
}

// Modern pattern form:
if (a instanceof Dog d) {
    d.bark();
}
```

`instanceof` performs a runtime type check before a cast. The pattern form both checks and creates a correctly typed variable inside the guarded scope.

12. Runtime Polymorphism

```
class Animal {
    void sound() { System.out.println("Animal"); }
}

class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog"); }
}

class Cat extends Animal {
    @Override
    void sound() { System.out.println("Cat"); }
}

void makeSound(Animal animal) {
    animal.sound();
}

makeSound(new Dog()); // Dog
makeSound(new Cat()); // Cat
```

A common parent type can accept many child objects. Each object supplies its own overridden behavior. This lets calling code depend on the general type instead of hard-coding every child type.

13. Field Access Trap

```
class Animal {
    String name = "Animal";
}

class Dog extends Animal {
    String name = "Dog";
}

Animal a = new Dog();
System.out.println(a.name); // Animal
```

Fields do not participate in runtime overriding the way instance methods do. The field access is resolved from the reference type. This is different from a virtual/overridden instance method call.

14. Static Method Trap

```
class Animal {
    static void info() { System.out.println("Animal info"); }
}

class Dog extends Animal {
    static void info() { System.out.println("Dog info"); }
}

Animal a = new Dog();
a.info(); // Animal info
```

Static methods are class-level. A same-signature static method in the child hides the parent method; this is not runtime overriding. Selection follows the reference/class context rather than the runtime object in the way an overridden instance method does.

15. super.member

```
class Animal {
    String name = "Animal";
    void sound() { System.out.println("Animal"); }
}

class Dog extends Animal {
    String name = "Dog";
    @Override
```

```

    void sound() { System.out.println("Dog"); }

    void test() {
        System.out.println(this.name);
        System.out.println(super.name);
        super.sound();
    }
}

```

this.name means the current object's field. super.name refers to the parent class field. super.sound() explicitly calls the parent implementation of the method.

16. Seven-Output Master Example

```

class Animal {
    String name = "Animal";
    void sound() { System.out.println("Animal sound"); }
    static void info() { System.out.println("Animal info"); }
}

class Dog extends Animal {
    String name = "Dog";
    @Override void sound() { System.out.println("Dog sound"); }
    static void info() { System.out.println("Dog info"); }
    void bark() { System.out.println("Bark"); }
}

Animal a = new Dog();
Dog d = (Dog) a;

System.out.println(a.name); // Animal
System.out.println(d.name); // Dog
a.sound();                 // Dog sound
d.sound();                 // Dog sound
a.info();                  // Animal info
d.info();                  // Dog info
d.bark();                  // Bark

```

17. Phase 3 Mental Model

Inheritance

- > Dog IS-A Animal
- > child gets access to inheritable parent members

Upcasting

- > Animal a = new Dog();

Compile time

- > reference type decides what members are accessible

Runtime

- > actual object decides overridden instance method implementation

Downcasting

- > Dog d = (Dog) a;
- > safe only when the actual object is a Dog-compatible object

instanceof

- > runtime type check

Fields

- > reference-type based access

Static methods

- > hidden, not runtime-overridden

18. Interview Checklist

1. Explain IS-A and HAS-A.
2. Explain constructor order in parent-child classes.
3. Difference between this() and super().

4. Overloading vs overriding.
5. Why use @Override?
6. What is upcasting?
7. Reference type vs actual object type.
8. Why can `Animal a = new Dog();` not call `bark()`?
9. What is downcasting? When can it fail?
10. What does `instanceof` do?
11. Why do overridden instance methods use runtime dispatch?
12. Why do fields behave differently from methods?
13. Why are static methods hidden instead of dynamically overridden?
14. Why can't constructors be overridden?
15. What does `super.sound()` mean?